

Claire System Book

ACS2 v1 -- END-TO-END SYSTEM FLOW

1. DATA INGESTION LAYER

****INPUT SOURCES:****

Product data (features, pricing, specs, reviews, returns, support logs)

Market data (search trends, sentiment, competitor catalogs, category growth)

Technology constraints (cost ceilings, materials, stack limits, regulatory rules)

****SUBSYSTEMS:****

ingestion/loaders.py -> load CSV/JSON/API data

ingestion/validators.py -> validate schema + required fields

ingestion/preprocess.py -> normalize into ProductInput objects

****OUTPUT:**** Normalized ProductInput records stored in DB

v

2. ANALYSIS LAYER (ML + PATTERN ENGINE)

****PURPOSE:**** Extract opportunity signals from real data

****SUBSYSTEMS:****

analysis/demand_model.py -> NLP sentiment, review embeddings, search signal scoring

analysis/pattern_engine.py -> unmet needs, feature gaps, underserved segments

analysis/feature_extractor.py -> structured feature vectors

scoring/demand.py -> demand score components

****OUTPUT:**** OpportunityPatterns (internal signal objects)

v

3. CONSTRAINED CONCEPT GENERATION (NEW PRODUCTS ONLY)

****PURPOSE:**** Generate NEW product concepts that are buildable now

****SUBSYSTEMS:****

concepts/generator.py -> recombine components, propose new configurations

concepts/constraints.py -> enforce cost, tech, regulatory, manufacturability limits

****EXECUTION ROUTE:****

ProductInput + OpportunityPatterns -> generator -> constraints -> Concept objects

****OUTPUT:**** Concept objects (new, not existing products)

v

4. VALIDATION LAYER (BUILDABILITY + DEMAND + DIFFERENTIATION + BREAKTHROUGH FILTER)

****PURPOSE:**** Reject anything unbuildable, unwanted, or non-differentiated

****SUBSYSTEMS:****

scoring/buildability.py -> manufacturability, cost, regulatory feasibility

scoring/demand.py -> consumer demand scoring

scoring/differentiation.py -> competitive gap scoring

concepts/breakthrough_filter.py -> strict threshold gating

****EXECUTION ROUTE:****

Concept -> scoring -> breakthrough_filter -> BreakthroughConcept

****OUTPUT:**** BreakthroughConcept objects

v

5. PORTFOLIO CREATION + MISSED DEAL DISCOVERY

****PURPOSE:**** Convert breakthrough concepts into portfolio assets and detect missed deals

****SUBSYSTEMS:****

portfolio/builder.py -> create PortfolioItem (concept + scores + evidence)

portfolio/evidence.py -> attach data references, charts, rationale

deals/discovery.py -> detect missed market opportunities

deals/deal_scorer.py -> score deal viability

deals/rationale.py -> generate strategic rationale

****EXECUTION ROUTE:****

BreakthroughConcept -> PortfolioItem

BreakthroughConcept -> DealPortfolioItem (missed deal)

****OUTPUT:**** PortfolioItem + DealPortfolioItem records in DB

v

6. ACQUIRER MATCHING + DESKTOP UI

****PURPOSE:**** Match portfolio items to best-fit acquirers and present everything in UI

****SUBSYSTEMS:****

acquirers/dataset.py -> curated acquirer dataset (public, safe)

acquirers/matcher.py -> sector fit, tech fit, capability gap fit

acquirers/scoring.py -> ranking + rationale

****API ROUTES:****

/concepts, /portfolio, /deals, /acquirers

****UI COMPONENTS:****

Dashboard.tsx

ConceptsView.tsx

PortfolioView.tsx

DealView.tsx

AcquirerMatches.tsx

****OUTPUT:**** RankedAcquirerMatches displayed in desktop UI

v

7. MONITORING + LEARNING LOOP

****PURPOSE:**** Improve ACS2 over time based on real outcomes

****SUBSYSTEMS:****

learning/feedback.py -> capture user accept/reject decisions

learning/metrics.py -> track build outcomes + acquirer engagement

learning/retraining.py -> adjust scoring models + thresholds

utils/logging.py -> system-wide event logging

****EXECUTION ROUTE:****

User actions + outcomes -> feedback -> metrics -> retraining -> improved models

****OUTPUT:**** A continuously improving ACS2 system

DETAILED SYSTEM FLOW

1. Application Launch

The user launches the desktop application. The Electron or Tauri frontend initializes. The backend service (FastAPI) is started or pinged to confirm it is running. System configuration, thresholds, and environment settings are loaded.

2. Data Ingestion

The user navigates to the Data Ingestion screen. The user selects product data files, market data files, and technology constraint files. The frontend sends these files to the backend ingestion endpoints. The ingestion loaders parse the files into raw records. The ingestion validators check schema correctness, required fields, and data integrity. The preprocessing module normalizes the data into ProductInput objects. The database layer stores ProductInput records in the local database.

3. Analysis Process

The user triggers the analysis process. The backend retrieves ProductInput records from the database. The feature extractor converts raw fields into structured feature vectors. The demand model analyzes reviews, sentiment, and search signals to estimate demand. The pattern engine identifies unmet needs, feature gaps, underserved segments, and rising demand clusters. The system produces internal OpportunityPatterns objects used by the concept generator.

4. Concept Generation

The system generates new product concepts. The concept generator receives ProductInput data and OpportunityPatterns. It recombines known components, proposes new configurations, and targets detected gaps. The constraints module enforces buildability rules, cost ceilings, regulatory limits, and manufacturability constraints. Only concepts that satisfy all constraints are instantiated as Concept objects. These Concept objects are stored or held in memory for scoring.

5. Validation

The system validates each concept. The buildability scoring module evaluates manufacturability, cost feasibility, technical feasibility, and regulatory feasibility. The demand scoring module evaluates consumer interest, sentiment, and market signals. The differentiation scoring module evaluates competitive gaps and uniqueness. The breakthrough filter applies strict thresholds to all scores. Concepts that pass all thresholds become BreakthroughConcept objects. Concepts that fail are discarded or marked as non-breakthrough.

6. Portfolio Creation

The system creates portfolio items and discovers missed deals. For each BreakthroughConcept, the portfolio builder creates a PortfolioItem containing concept details, scores, evidence references, and rationale. The evidence module links portfolio items to underlying data sources. The deal discovery module analyzes market gaps, competitor blind spots, and strategic opportunities. The deal scoring module evaluates the viability of each potential deal. The rationale module generates structured explanations for each deal opportunity. The system creates DealPortfolioItem objects representing missed real-world opportunities. The portfolio repository stores PortfolioItem and DealPortfolioItem records in the database.

7. Acquirer Matching

The system performs acquirer matching. The acquirer dataset module loads a curated list of potential acquirers with sector, strategy, tech focus, and capability gap metadata. The matcher module evaluates fit between each portfolio item or deal item and each acquirer. The scoring module computes sector fit, tech fit, capability fit, and strategic fit. The system produces MatchResult objects containing ranked acquirers and rationale. These results are stored in the database and exposed through API endpoints.

8. UI Display

The desktop UI displays all results. The frontend calls the backend API to retrieve concepts, portfolio items, deal items, and acquirer matches. The Dashboard screen shows system metrics and counts. The Concepts View shows generated concepts and their validation status. The Portfolio View shows portfolio items with evidence and scores. The Deal View shows missed deal opportunities. The Acquirer Matches View shows ranked acquirers and rationale. The user can inspect, filter, and export results.

9. User Feedback

The user provides feedback. The user can accept or reject concepts. The user can mark portfolio items as built, in pilot, or dropped. The user can mark acquirer matches as contacted, interested, not interested, or no response. The frontend sends this feedback to the backend feedback endpoints. The feedback module stores these decisions in the database.

10. Learning Loop

The system learns from outcomes. The metrics module aggregates performance data such as acceptance rates, build success rates, and acquirer engagement. The retraining module uses this data to adjust thresholds, refine scoring models, and improve deal discovery and matching logic. Over time, the system becomes more accurate and aligned with real-world outcomes.

11. Continuous Operation

The system is ready for repeated cycles. The user can ingest new data, rerun analysis, generate new concepts, update the portfolio, discover new deals, and match new acquirers. The system maintains a persistent history of concepts, portfolio items, deals, matches, and feedback. The

desktop application functions as a self-contained intelligence engine for product and deal discovery.

SYSTEM FLOW DIAGRAM

1. DATA INGESTION Load product data (features, pricing, specs, reviews, returns, support logs) Load market data (search trends, sentiment, competitor catalogs, category growth) Load technology constraints (cost ceilings, materials, stack limits, regulatory rules) Validate raw files Normalize into ProductInput objects Store ProductInput records in database v 2. ANALYSIS (ML AND PATTERN ENGINE) Extract structured features Run demand modeling (NLP, sentiment, search signals) Detect unmet needs Detect feature gaps Detect underserved segments Detect rising demand clusters Produce OpportunityPatterns v 3. CONSTRAINED CONCEPT GENERATION Use ProductInput + OpportunityPatterns Recombine known components Propose new configurations Target detected opportunity patterns Enforce buildability constraints Enforce cost constraints Enforce regulatory constraints Enforce manufacturability constraints Produce new Concept objects v 4. VALIDATION (BUILDABILITY, DEMAND, DIFFERENTIATION, BREAKTHROUGH) Score buildability Score demand Score differentiation Apply breakthrough thresholds Reject non-qualifying concepts Promote qualifying concepts to BreakthroughConcept v 5. PORTFOLIO CREATION AND MISSED DEAL DISCOVERY Convert BreakthroughConcept -> PortfolioItem Attach evidence (data references, charts, rationale) Detect missed market opportunities Detect competitor blind spots Detect strategic gaps Score deal viability Generate deal rationale Create DealPortfolioItem v 6. ACQUIRER MATCHING AND DESKTOP UI Load acquirer dataset Compute sector fit Compute tech fit Compute capability fit Compute strategic fit Rank acquirers Produce MatchResult objects Display concepts in UI Display portfolio items in UI Display deal opportunities in UI Display acquirer matches in UI v 7. MONITORING AND LEARNING LOOP Capture user feedback (accept, reject, built, pilot, dropped) Capture acquirer engagement (interested, not interested, no response) Aggregate performance metrics Adjust thresholds Retrain scoring models Improve deal discovery logic Improve acquirer matching logic Improve breakthrough filtering

BACKEND API MAP -- ACS2 v1 End-to-End API Surface

****API ROOT****

Base URL: <http://localhost:8000>

Purpose: Local backend service powering the ACS2 desktop application.

1. INGESTION API

Purpose: Accept product, market, and tech data files and load them into the system.

****Endpoints:****

POST /ingestion/products - Upload product, market, or tech data files. Body: multipart file + type field. Result: records ingested count

GET /ingestion/summary - Returns counts of ingested product, market, and tech records.

****Used by:**** Data Ingestion UI, Analysis stage (pulls ProductInput from DB)

2. CONCEPT GENERATION API

Purpose: Trigger analysis, concept generation, and validation.

****Endpoints:****

POST /concepts/generate - Runs analysis -> concept generation -> scoring -> breakthrough filter. Returns: number of generated concepts and number of breakthrough concepts.

GET /concepts - Returns all concepts with scores and breakthrough status.

GET /concepts/{concept_id} - Returns details for a single concept.

****Used by:**** Concepts View, Portfolio creation stage, Deal discovery stage

3. PORTFOLIO API

Purpose: Manage portfolio items created from breakthrough concepts.

****Endpoints:****

GET /portfolio - Returns all PortfolioItem records.

GET /portfolio/{portfolio_id} - Returns a single PortfolioItem with evidence and rationale.

PATCH /portfolio/{portfolio_id}/status - Update status: candidate, accepted, rejected, built, in_pilot.

****Used by:**** Portfolio View, Acquirer matching, Learning loop

4. DEAL DISCOVERY API

Purpose: Expose missed deal opportunities discovered from breakthrough concepts.

****Endpoints:****

GET /deals - Returns all DealPortfolioItem records.

GET /deals/{deal_id} - Returns a single DealPortfolioItem.

PATCH /deals/{deal_id}/status - Update deal status: active or archived.

****Used by:**** Deal View, Acquirer matching, Learning loop

5. ACQUIRER MATCHING API

Purpose: Provide acquirer match results for portfolio items and deals.

****Endpoints:****

GET /acquirers - Returns the full acquirer dataset.

GET /acquirers/matches - Returns match results for all portfolio items.

GET /portfolio/{portfolio_id}/acquirers - Returns match results for a specific PortfolioItem.

GET /deals/{deal_id}/acquirers - Returns match results for a specific DealPortfolioItem.

****Used by:**** Acquirer Matches View, Strategy evaluation, Learning loop

6. FEEDBACK API

Purpose: Capture user decisions and real-world outcomes for the learning loop.

****Endpoints:****

POST /feedback/concept - Accept or reject a concept.

POST /feedback/portfolio - Mark portfolio item outcome: built, not_built, in_pilot.

POST /feedback/acquirer - Record acquirer engagement: interested, not_interested, no_response.

****Used by:**** Monitoring and learning loop, Threshold adjustment, Model retraining

7. SYSTEM HEALTH API

Purpose: Ensure backend is running.

****Endpoints:****

GET /health - Returns: status ok

****Used by:**** Desktop app startup, Continuous backend monitoring

DATA MODEL MAP -- ACS2 v1 Full System Data Structures

PRODUCTINPUT (INGESTION STAGE)

Purpose: Normalized representation of raw product, market, and tech data.

****Fields:**** id, source_type (product, market, tech), raw_data (dict), normalized_fields (dict), created_at

****Used by:**** Analysis, Concept Generation

OPPORTUNITYPATTERNS (ANALYSIS STAGE)

Purpose: Structured signals extracted from ProductInput.

Fields: id, unmet_needs (list), feature_gaps (list), underserved_segments (list), demand_clusters (list), extracted_features (dict)

Used by: Concept Generation

CONCEPT (GENERATION STAGE)

Purpose: New product idea generated from data.

Fields: id, name, description, source_product_ids (list of ProductInput ids), opportunity_pattern_ids (list of OpportunityPatterns ids), config (dict of components/features), created_at

Used by: Scoring, Breakthrough Filter, Portfolio Builder, Deal Discovery

SCORES (VALIDATION STAGE)

Purpose: Quantitative evaluation of a Concept.

Fields: concept_id, buildability_score, demand_score, differentiation_score, composite_score, details (dict of sub-scores)

Used by: Breakthrough Filter, Portfolio Builder, Deal Scorer

BREAKTHROUGHCONCEPT (POST-VALIDATION)

Purpose: Concept that passed all scoring thresholds.

Fields: concept_id, scores (Scores object), breakthrough_reasoning (optional)

Used by: Portfolio Builder, Deal Discovery

PORTFOLIOITEM (PORTFOLIO STAGE)

Purpose: A fully validated, evidence-backed product opportunity.

Fields: id, concept_id, title, summary, scores (Scores), evidence_refs (list of strings), rationale (string), status (candidate, accepted, rejected, built, in_pilot), created_at, updated_at

Used by: Deal Discovery, Acquirer Matching, Learning Loop

DEALPORTFOLIOITEM (MISSED DEAL DISCOVERY STAGE)

Purpose: A real-world deal opportunity derived from a breakthrough concept.

****Fields:**** id, concept_id, portfolio_item_id, deal_title, deal_summary, deal_score, strategic_rationale, market_gap_description, status (active, archived), created_at, updated_at

****Used by:**** Acquirer Matching, Learning Loop

ACQUIRER (MATCHING STAGE)

Purpose: Representation of a potential acquiring company.

****Fields:**** id, name, sector, size (startup, mid, enterprise), geography, tech_focus (list), strategy_tags (list), capability_gaps (list), notes

****Used by:**** Matcher, Scoring, UI

MATCHRESULT (MATCHING OUTPUT)

Purpose: Ranked match between a portfolio item or deal and an acquirer.

****Fields:**** id, portfolio_item_id (nullable), deal_portfolio_item_id (nullable), acquirer_id, fit_score, sector_fit, tech_fit, capability_fit, strategic_fit, rationale, created_at

****Used by:**** UI, Learning Loop

FEEDBACK MODELS (LEARNING LOOP)

****ConceptFeedback:**** id, concept_id, decision (accepted, rejected), reason, created_at

****PortfolioFeedback:**** id, portfolio_item_id, outcome (built, not_built, in_pilot), created_at

****AcquirerFeedback:**** id, match_result_id, engagement (interested, not_interested, no_response), created_at

****Used by:**** Metrics, Retraining, Threshold Adjustment

SUBSYSTEM DEPENDENCY GRAPH -- ACS2 v1

End-to-End Architectural Dependencies

INGESTION SUBSYSTEM

****Modules:**** ingestion/loaders, ingestion/validators, ingestion/preprocess, ProductInput schema

****Depends on:**** None (entry point of system)

****Used by:**** Analysis subsystem, Concept Generation subsystem

ANALYSIS SUBSYSTEM

Modules: analysis/feature_extractor, analysis/demand_model, analysis/pattern_engine, scoring/demand

Depends on: Ingestion subsystem (ProductInput data), Database

Used by: Concept Generation subsystem, Scoring subsystem

CONCEPT GENERATION SUBSYSTEM

Modules: concepts/generator, concepts/constraints, Concept schema

Depends on: Analysis subsystem (OpportunityPatterns), Ingestion subsystem (ProductInput)

Used by: Scoring subsystem, Breakthrough Filter, Portfolio subsystem, Deal Discovery subsystem

SCORING SUBSYSTEM

Modules: scoring/buildability, scoring/demand, scoring/differentiation, scoring_utils, Scores schema

Depends on: Concept Generation subsystem, Analysis subsystem

Used by: Breakthrough Filter, Portfolio subsystem, Deal Discovery subsystem

BREAKTHROUGH FILTER

Modules: concepts/breakthrough_filter

Depends on: Scoring subsystem

Used by: Portfolio subsystem, Deal Discovery subsystem

PORTFOLIO SUBSYSTEM

Modules: portfolio/builder, portfolio/evidence, portfolio/repository, PortfolioItem schema

Depends on: Breakthrough Filter, Scoring subsystem, Concept Generation subsystem

Used by: Deal Discovery subsystem, Acquirer Matching subsystem, Learning subsystem, UI

DEAL DISCOVERY SUBSYSTEM

Modules: deals/discovery, deals/deal_scorer, deals/rationale, DealPortfolioItem schema

Depends on: Breakthrough Filter, Portfolio subsystem, Scoring subsystem

Used by: Acquirer Matching subsystem, Learning subsystem, UI

ACQUIRER MATCHING SUBSYSTEM

****Modules:**** acquirers/dataset, acquirers/matcher, acquirers/scoring, Acquirer schema, MatchResult schema

****Depends on:**** Portfolio subsystem, Deal Discovery subsystem

****Used by:**** UI, Learning subsystem

API SUBSYSTEM

****Modules:**** routes_ingestion, routes_concepts, routes_portfolio, routes_deals, routes_acquirers, routes_feedback

****Depends on:**** All backend subsystems

****Used by:**** Desktop UI

LEARNING SUBSYSTEM

****Modules:**** learning/feedback, learning/metrics, learning/retraining

****Depends on:**** Portfolio subsystem, Deal Discovery subsystem, Acquirer Matching subsystem, User feedback via API

****Used by:**** Scoring subsystem (model updates), Breakthrough Filter (threshold updates), Deal Discovery subsystem (improved scoring), Acquirer Matching subsystem (improved fit logic)

This comprehensive system book demonstrates Claire AI's complete end-to-end architecture, from data ingestion through learning loops, providing a sovereign, deterministic, and modular intelligence engine ready for enterprise and federal deployment.